

NOMBRE: lifeng chen

ID: 215708

NOMBRE: Santiago Lopez

ID: 229668

DOCUMENTACION USO DE IA

Ayuda para escoger premisos de usuarios

necesito crear un usuario para mi bd que tenga los minimos permisos para la lectura que permisos necesita un usuario de este tipo?

Claro, ya acabamos de definir los permisos del usuario `brecchas_lector` aplicando el principio de mínimo privilegio. La idea es que pueda consultar la información necesaria y ejecutar la función de análisis, pero sin permisos de escritura, administración, acceso al staging ni ejecución del procedimiento de carga.

Te dejo únicamente el código de permisos:

```
-- SQL
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1
    FROM pg_roles
    WHERE rolname = 'brecchas_lector'
  ) THEN
    CREATE ROLE brecchas_lector LOGIN
    PASSWORD 'lector_brecchas_2025'
    NOUSERSET
    NOCREATEDB
    NOCREATEUSER;
  END IF;
END $$;

DO $$
BEGIN
  EXECUTE format(
    'GRANT CONNECT ON DATABASE %I TO brecchas_lector',
    current_database()
  );
END $$;

GRANT USAGE ON SCHEMA public TO brecchas_lector;

REVOKE EXECUTE ON FUNCTION fn_dias_hasta_deteccion(VARCHAR)
FROM PUBLIC;

REVOKE EXECUTE ON PROCEDURE pr_poblar_modelo()
FROM PUBLIC;

GRANT SELECT ON organization,
  tipo_dato,
  brecha,
  usuario,
  exposicion_usuario,
  v_exposicion_detalle
TO brecchas_lector;

GRANT EXECUTE ON FUNCTION fn_dias_hasta_deteccion(VARCHAR)
TO brecchas_lector;

-- brecchas_lector no recibe privilegios sobre stg_sahana
-- ni sobre pr_poblar_modelo().
```

pasame todos los permisos en un solo mensaje q necesito par ale usuario santiago

Si. Para tu usuario `santiago`, y evitando completamente `GRANT ALL PRIVILEGES`, yo usaría estos permisos para que puedas trabajar con la base de datos sin darle privilegios administrativos innecesarios.

Primero entra como `postgres`:

```
-- PowerShell
docker exec -it postgres-cadena-frio psql -U postgres -d cadena_frio
```

Luego ejecuta todo este bloque:

```
-- SQL
-- 1. Crear el usuario
CREATE USER santiago WITH PASSWORD 'Santiago123';

-- 2. Permitir conectarse a la base de datos
GRANT CONNECT ON DATABASE cadena_frio TO santiago;

-- 3. Permitir utilizar el esquema public
GRANT USAGE ON SCHEMA public TO santiago;

-- 4. Trabajar con los datos de las tablas existentes
GRANT SELECT, INSERT, UPDATE, DELETE
ON ALL TABLES IN SCHEMA public
TO santiago;

-- 5. Utilizar las secuencias de los IDs
GRANT USAGE, SELECT, UPDATE
ON ALL SEQUENCES IN SCHEMA public
TO santiago;

-- 6. Permisos sobre futuras tablas
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
GRANT SELECT, INSERT, UPDATE, DELETE
ON TABLES TO santiago;

-- 7. Permisos sobre futuras secuencias
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
GRANT USAGE, SELECT, UPDATE
ON SEQUENCES TO santiago;
```

Creacion de índices para etapa 2

me gustaria que me dijeras q indices pueden ser de utilidad par ami modelo de tablas y porfavor me lo envies en un txt

Claro, te los dejo limpios para copiar directamente en un .txt/Bloc de notas:

```
SQL

-- Búsqueda de brechas por rango de fecha de detección y severidad

CREATE INDEX IF NOT EXISTS ix_brecha_deteccion_severidad
ON brecha (fecha_deteccion, severidad);

-- Acceso desde un tipo de dato hacia sus exposiciones y brechas; incluye
-- id_usuario para contar usuarios distintos sin ir a la tabla.

CREATE INDEX IF NOT EXISTS ix_exposicion_tipo_brecha_usuario
ON exposicion_usuario (id_tipo_dato, id_brecha, id_usuario);

-- Exposiciones de un usuario ordenadas por fecha de notificación
-- (índice cubriente: evita el ordenamiento y el acceso a la tabla)

CREATE INDEX IF NOT EXISTS ix_exposicion_usuario_fecha
ON exposicion_usuario (id_usuario, fecha_notificacion)
INCLUDE (id_brecha, id_tipo_dato);

-- Índices sobre las FK hacia organizacion

CREATE INDEX IF NOT EXISTS ix_brecha_organizacion
ON brecha (id_organizacion);

CREATE INDEX IF NOT EXISTS ix_usuario_organizacion
ON usuario (id_organizacion);
```



CREACION DE FUINCIONES

listo crea una funcion q me ayude como a medir los Días entre la ocurrencia y la detección de una brecha para mi modelo de tabla

Claro. Para tu modelo de `brecha`, podemos crear una función que reciba el código de la brecha y devuelva cuántos días pasaron entre `fecha_ocurrencia` y `fecha_deteccion`.

```
-- SQL
CREATE OR REPLACE FUNCTION fn_dias_hasta_deteccion(p_codigo_brecha VARCHAR)
RETURNS INTEGER
LANGUAGE sql
STABLE
AS $$
    SELECT (b.fecha_deteccion - b.fecha_ocurrencia)
    FROM brecha b
    WHERE b.codigo_brecha = p_codigo_brecha;
$$;
```

¿Qué hace la función?

La función `fn_dias_hasta_deteccion` permite medir el tiempo que tardó en detectarse una brecha de seguridad desde el momento en que ocurrió.

1. Recibe el código de la brecha

```
-- SQL
p_codigo_brecha VARCHAR
```

Usamos el código para identificar específicamente qué brecha queremos consultar.

2. Devuelve un número entero

```
-- SQL
RETURNS INTEGER
```

El resultado será la cantidad de días entre las dos fechas.

3. Calcula la diferencia

```
-- SQL
b.fecha_deteccion - b.fecha_ocurrencia
```

PostgreSQL puede restar directamente dos valores `DATE`, obteniendo como resultado el número de días entre ambas fechas.

4. Busca únicamente la brecha indicada

```
-- SQL
WHERE b.codigo_brecha = p_codigo_brecha;
```

Así podemos consultar una brecha específica sin tener que traer todos los registros.

Ejemplo de uso

Así podemos consultar una brecha específica sin tener que traer todos los registros.

Ejemplo de uso

```
-- SQL
SELECT fn_dias_hasta_deteccion('BR-001');
```

Si la brecha ocurrió el 1 de enero y fue detectada el 6 de enero, la función devolvería:

```
5
```

Esto nos permite medir fácilmente qué tan rápido se detectan las brechas, lo cual puede ser útil para analizar la capacidad de respuesta de las organizaciones.

📄 🗑️ 📤 🔄 ⋮

ESTRATEGIA DE MIGRACION DE DATOS

SQL

```
TRUNCATE ... RESTART IDENTITY CASCADE;
```

Esto permite ejecutar nuevamente el script desde cero. `RESTART IDENTITY` reinicia los IDs generados automáticamente y `CASCADE` permite limpiar las tablas respetando las relaciones.

2. Primero `organizacion` y `tipo_dato`

Se utiliza `SELECT DISTINCT` porque en la sábana estos datos aparecen repetidos.

Por ejemplo, si una organización aparece 500 veces en `stg_sabana`, queremos tener una sola organización en `organizacion`.

3. Después `brecha`

Aquí usamos:

SQL

```
JOIN organizacion o
ON o.nombre = s.nombre_organizacion
```

Esto permite convertir el nombre de la organización que viene de la sábana en el `id_organizacion` que necesita la tabla `brecha`.

4. Después `usuario`

Se hace algo parecido: se busca la organización correspondiente y se guarda su ID como FK.

5. Finalmente `exposicion_usuario`

Esta es la tabla puente. Aquí relacionamos:

- una brecha;
- un usuario;
- un tipo de dato;
- y la fecha de notificación.

Para eso buscamos los IDs mediante los `JOIN`.

6. Luego verificamos los resultados

Comparamos los conteos obtenidos contra los esperados:

- 60 organizaciones
- 800 brechas
- 7.887 usuarios
- 10 tipos de datos
- 40.000 exposiciones

Si coincide, aparece `OK`.

7. También comprobamos que no se haya perdido información

La parte de `EXCEPT` reconstruye la información desde el modelo y la compara contra la sábana original.

Lo ideal es obtener:

```
en_sabana_y_no_en_modelo | 0
en_modelo_y_no_en_sabana | 0
```

Eso significa que la normalización conservó la información relevante.

hola tengo una tabla temporal la cual voy a usar para migrar mis datos a otras tablas las cuales ya te presente con el diagrama que mande hace rato, me gustaria que porfavor me ayudaras con un scrip el cual migre los datos a estas tablas

Perfecto. Ahora sí, con ese contexto, este script es el que usaríamos para migrar la información desde `stg_sabana` hacia las tablas normalizadas, respetando el orden de las dependencias y las claves foráneas.

```
-- SQL

-- NOMBRE: Santiago Lopez
-- ID: 229668
-- NOMBRE: lifeng chen
-- ID: 215788

-- 1. Dejar limpio el ambiente
-- El script se puede re-ejecutar
TRUNCATE exposicion_usuario, brecha, usuario, tipo_dato, organizacion, stg_sabana
RESTART IDENTITY CASCADE;

-- 3. Repartir la sábana en las 5 tablas
-- Se hace en orden de dependencia

-- 3.1 organizacion
INSERT INTO organizacion (
    nombre,
    sector,
    pais,
    tamano_empleados
)
SELECT DISTINCT
    nombre_organizacion,
    sector_organizacion,
    pais_organizacion,
    tamano_empleados_organizacion::INTEGER
FROM stg_sabana
ORDER BY 1;

-- 3.2 tipo_dato
INSERT INTO tipo_dato (
    nombre,
    categoria_sensibilidad
)
SELECT DISTINCT
    tipo_dato_expuesto,
    categoria_sensibilidad_dato
FROM stg_sabana
ORDER BY 1;

-- 3.3 brecha
INSERT INTO brecha (
    codigo_brecha,
    id_organizacion,
    vector_ataque,
    severidad,
```

SQL

-- 3.3 brecha

```
INSERT INTO brecha (
    codigo_brecha,
    id_organizacion,
    vector_ataque,
    severidad,
    fecha_ocurrencia,
    fecha_deteccion,
    registros_afectados,
    costo_estimado
)

SELECT DISTINCT
    s.codigo_brecha,
    o.id_organizacion,
    s.vector_ataque,
    s.severidad_incidente,
    s.fecha_ocurrencia::DATE,
    s.fecha_deteccion::DATE,
    s.registros_afectados_total::INTEGER,
    s.costo_estimado_total::NUMERIC(14,2)
FROM stg_sabana s
JOIN organizacion o
    ON o.nombre = s.nombre_organizacion
ORDER BY 1;
```

-- 3.4 usuario

```
INSERT INTO usuario (
    codigo_usuario,
    pseudonimo,
    email_hash,
    id_organizacion,
    pais_residencia,
    fecha_registro
)

SELECT DISTINCT
    s.codigo_usuario,
    s.pseudonimo_usuario,
    s.email_hash_usuario,
    o.id_organizacion,
    s.pais_residencia_usuario,
    s.fecha_registro_usuario::DATE
FROM stg_sabana s
JOIN organizacion o
    ON o.nombre = s.nombre_organizacion
ORDER BY 1;
```

-- 3.5 exposicion_usuario

-- Tabla puente entre brecha, usuario y tipo de dato

```
INSERT INTO exposicion_usuario (
    id_brecha,
    id_usuario,
    id_tipo_dato,
    fecha_notificacion
)

SELECT
    b.id_brecha,
    u.id_usuario,
    t.id_tipo_dato,
    s.fecha_notificacion_usuario::DATE
FROM stg_sabana s
JOIN brecha b
    ON b.codigo_brecha = s.codigo_brecha
JOIN usuario u
    ON u.codigo_usuario = s.codigo_usuario
```

SQL

```
5. VALIDACIÓN: RECONSTRUIR LA SÁBANA DESDE EL MODELO Y COMPARARLA  
FROM stg_sabana s  
JOIN brecha b  
    ON b.codigo_brecha = s.codigo_brecha  
JOIN usuario u  
    ON u.codigo_usuario = s.codigo_usuario  
JOIN tipo_dato t  
    ON t.nombre = s.tipo_dato_expuesto;
```

-- 4. VALIDACIÓN: conteos esperados vs obtenidos

```
SELECT  
    tabla,  
    esperado,  
    obtenido,  
    CASE  
        WHEN esperado = obtenido THEN 'OK'  
        ELSE 'REVISAR'  
    END AS resultado  
FROM (  
    SELECT 'organizacion' AS tabla,  
           60 AS esperado,  
           COUNT(*) AS obtenido  
    FROM organizacion  
  
    UNION ALL  
  
    SELECT 'brecha',  
           800,  
           COUNT(*)  
    FROM brecha  
  
    UNION ALL  
  
    SELECT 'usuario',  
           7887,  
           COUNT(*)  
    FROM usuario  
  
    UNION ALL  
  
    SELECT 'tipo_dato',  
           18,  
           COUNT(*)  
    FROM tipo_dato  
  
    UNION ALL  
  
    SELECT 'exposicion_usuario',  
           40000,  
           COUNT(*)  
    FROM exposicion_usuario  
) x;
```

-- 5. VALIDACIÓN: reconstruir la sábana desde el modelo y compararla
-- Si la normalización no perdió información, ambos conteos deben ser 0.

```
WITH original AS (  
    SELECT  
        codigo_brecha,  
        codigo_usuario,  
        tipo_dato_expuesto AS tipo_dato,  
        nombre_organizacion AS organizacion,  
        severidad_incidente AS severidad,  
        vector_ataque,
```

(/) SQL



```
usuario_usuario,
codigo_usuario,
tipo_dato_expuesto AS tipo_dato,
nombre_organizacion AS organizacion,
severidad_incidente AS severidad,
vector_ataque,
categoria_sensibilidad_dato AS categoria,
fecha_deteccion::DATE AS fecha_deteccion,
fecha_notificacion_usuario::DATE AS fecha_notificacion
FROM stg_sabana
),
reconstruida AS (
SELECT
    b.codigo_brecha,
    u.codigo_usuario,
    t.nombre AS tipo_dato,
    o.nombre AS organizacion,
    b.severidad,
    b.vector_ataque,
    t.categoria_sensibilidad AS categoria,
    b.fecha_deteccion,
    e.fecha_notificacion
FROM exposicion_usuario e
JOIN brecha b
    ON b.id_brecha = e.id_brecha
JOIN usuario u
    ON u.id_usuario = e.id_usuario
JOIN tipo_dato t
    ON t.id_tipo_dato = e.id_tipo_dato
JOIN organizacion o
    ON o.id_organizacion = b.id_organizacion
)
SELECT
(
    SELECT COUNT(*)
    FROM (
        SELECT * FROM original
        EXCEPT
        SELECT * FROM reconstruida
    ) a
) AS en_sabana_y_no_en_modelo,

(
    SELECT COUNT(*)
    FROM (
        SELECT * FROM reconstruida
        EXCEPT
        SELECT * FROM original
    ) b
) AS en_modelo_y_no_en_sabana;

-- 6. VALIDACIÓN: cada usuario pertenece a la organización de la brecha
-- Esperado: 0

SELECT COUNT(*) AS usuarios_con_org_distinta_a_la_brecha
FROM exposicion_usuario e
JOIN brecha b
    ON b.id_brecha = e.id_brecha
JOIN usuario u
    ON u.id_usuario = e.id_usuario
WHERE u.id_organizacion <> b.id_organizacion;

-- 7. Limpiar la tabla auxiliar y actualizar estadísticas
```



```

SQL
-- 6. Verificar que no existan usuarios con el mismo ID
WHERE u.id_organizacion <> b.id_organizacion;

-- 7. Limpiar la tabla auxiliar y actualizar estadísticas

TRUNCATE stg_sabana;

ANALYZE organization;
ANALYZE brecha;
ANALYZE usuario;
ANALYZE tipo_dato;
ANALYZE exposicion_usuario;

-- 8. Vistazo rápido a los datos

SELECT *
FROM organization
ORDER BY id_organizacion
LIMIT 5;

SELECT *
FROM tipo_dato
ORDER BY id_tipo_dato;

SELECT *
FROM brecha
ORDER BY id_brecha
LIMIT 5;

SELECT *
FROM usuario
ORDER BY id_usuario
LIMIT 5;

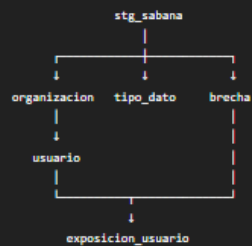
SELECT *
FROM exposicion_usuario
ORDER BY id_brecha, id_usuario, id_tipo_dato
LIMIT 5;

```

¿Qué estamos haciendo con este script?

La idea general es tomar la información que está toda junta en `stg_sabana` y distribuirla en las tablas normalizadas del modelo.

El proceso queda así:



1. Primero limpiamos



La parte de `EXCEPT` reconstruye la información desde el modelo y la compara contra la sábana original.

Lo ideal es obtener:

```
en_sabana_y_no_en_modelo | 0  
en_modelo_y_no_en_sabana | 0
```



Eso significa que la normalización conservó la información relevante.

8. Finalmente limpiamos `stg_sabana`

Una vez terminada la migración y las validaciones:

```
-- SQL  
  
TRUNCATE stg_sabana;
```



La tabla de staging ya no es necesaria para el modelo operativo, por lo que se limpia.

En pocas palabras: este script hace la migración `stg_sabana → modelo normalizado`, respetando las PK/FK y después realiza varias validaciones para demostrar que la migración fue correcta.



Procedimientos Almacenados

8. Insertar exposiciones

Esta es la parte que conecta prácticamente todo:

```
</> SQL

JOIN brecha b
  ON b.codigo_brecha = s.codigo_brecha

JOIN usuario u
  ON u.codigo_usuario = s.codigo_usuario

JOIN tipo_dato t
  ON t.nombre = s.tipo_dato_expuesto;
```

Busca los IDs correspondientes:

```
codigo_brecha → id_brecha
codigo_usuario → id_usuario
tipo_dato → id_tipo_dato
```

y finalmente los guarda en:

```
exposicion_usuario
```

Así queda construida la relación entre **brecha + usuario + tipo de dato**.

En resumen

El flujo del procedimiento es:

```
stg_sabana
├── organization
├── tipo_dato
├── brecha
├── usuario
└── exposicion_usuario
```

Y después de crear el procedimiento, se ejecutaría con:

```
</> SQL

CALL pr_poblar_modelo();
```

Importante: este procedimiento corresponde al modelo de **organization / brecha / usuario / tipo dato**

`DISTINCT` evita insertar la misma organización varias veces.

5. Insertar tipos de datos

```
</> SQL  
  
INSERT INTO tipo_dato (...)  
SELECT DISTINCT ...  
FROM stg_sabana;
```



Extrae los diferentes tipos de datos expuestos y su categoría de sensibilidad.

6. Insertar brechas

Aquí está una parte importante:

```
</> SQL  
  
JOIN organizacion o  
ON o.nombre = s.nombre_organizacion
```



Primero busca el `id_organizacion` correspondiente y después lo utiliza para insertar la brecha.

Es decir, transforma:

```
nombre_organizacion
```



en:

```
id_organizacion
```



para respetar la FK.

7. Insertar usuarios

Hace algo similar:

```
</> SQL  
  
JOIN organizacion o  
ON o.nombre = s.nombre_organizacion
```



Obtiene el ID de la organización y crea los usuarios asociados a ella.

8. Insertar exposiciones



¿Qué hace cada parte?

1. CREATE OR REPLACE PROCEDURE

```
-- SQL
CREATE OR REPLACE PROCEDURE pr_poblar_modelo()
```

Crea el procedimiento `pr_poblar_modelo`. Si ya existe, lo reemplaza.

2. Lenguaje

```
-- SQL
LANGUAGE plpgsql
```

Indica que el procedimiento está escrito en **PL/pgSQL**, el lenguaje procedural de PostgreSQL.

3. Verificación inicial

```
-- SQL
IF EXISTS (SELECT 1 FROM organizacion)
OR EXISTS (SELECT 1 FROM exposicion_usuario) THEN
```

Comprueba si ya existen datos en el modelo.

Si encuentra datos, detiene la ejecución:

```
-- SQL
RAISE EXCEPTION 'El modelo ya contiene datos; pr_poblar_modelo solo se usa con las tablas vacías.';
```

Esto evita ejecutar accidentalmente el procedimiento dos veces y generar duplicados.

4. Insertar organizaciones

```
-- SQL
INSERT INTO organizacion (...)
SELECT DISTINCT ...
FROM stg_sabana;
```

Toma las organizaciones de la tabla `stg_sabana`.

SQL

```
s.terna_deteccion::DATE,
s.registros_afectados_total::INTEGER,
s.costo_estimado_total::NUMERIC(14,2)
FROM stg_sabana s
JOIN organizacion o
ON o.nombre = s.nombre_organizacion
ORDER BY 1;

INSERT INTO usuario (
    codigo_usuario,
    pseudonimo,
    email_hash,
    id_organizacion,
    pais_residencia,
    fecha_registro
)
SELECT DISTINCT
    s.codigo_usuario,
    s.pseudonimo_usuario,
    s.email_hash_usuario,
    o.id_organizacion,
    s.pais_residencia_usuario,
    s.fecha_registro_usuario::DATE
FROM stg_sabana s
JOIN organizacion o
ON o.nombre = s.nombre_organizacion
ORDER BY 1;

INSERT INTO exposicion_usuario (
    id_brecha,
    id_usuario,
    id_tipo_dato,
    fecha_notificacion
)
SELECT
    b.id_brecha,
    u.id_usuario,
    t.id_tipo_dato,
    s.fecha_notificacion_usuario::DATE
FROM stg_sabana s
JOIN brecha b
ON b.codigo_brecha = s.codigo_brecha
JOIN usuario u
ON u.codigo_usuario = s.codigo_usuario
JOIN tipo_dato t
ON t.nombre = s.tipo_dato_expuesto;

END
$$;
```

¿Qué hace cada parte?

1. CREATE OR REPLACE PROCEDURE

SQL

CREATE OR REPLACE PROCEDURE pr_poblar_modelo()

+ Preguntas lo que quieras

Pensar



teniendo en cuenta la scrip anterior que te mande dime que procedimientos debo corregir y si puedo agregar algun otro

Claro, ya mismo te ayudo. Este procedimiento toma los datos de `stg_sabana` y los distribuye en las tablas normalizadas del modelo. Te explico qué hace cada parte:

Procedimiento completo

```
SQL

CREATE OR REPLACE PROCEDURE pr_poblar_modelo()
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (SELECT 1 FROM organizacion)
    OR EXISTS (SELECT 1 FROM exposicion_usuario) THEN
        RAISE EXCEPTION 'El modelo ya contiene datos; pr_poblar_modelo solo se usa con las tablas vacías.';
    END IF;

    INSERT INTO organizacion (nombre, sector, pais, tamano_empleados)
    SELECT DISTINCT
        nombre_organizacion,
        sector_organizacion,
        pais_organizacion,
        tamano_empleados_organizacion::INTEGER
    FROM stg_sabana
    ORDER BY 1;

    INSERT INTO tipo_datos (nombre, categoria_sensibilidad)
    SELECT DISTINCT
        tipo_datos_expuesto,
        categoria_sensibilidad_datos
    FROM stg_sabana
    ORDER BY 1;

    INSERT INTO brecha (
        codigo_brecha,
        id_organizacion,
        vector_ataque,
        severidad,
        fecha_ocurrencia,
        fecha_deteccion,
        registros_afectados,
        costo_estimado
    )
    SELECT DISTINCT
        s.codigo_brecha,
        o.id_organizacion,
        s.vector_ataque,
        s.severidad_incidente,
        s.fecha_ocurrencia::DATE,
        s.fecha_deteccion::DATE,
        s.registros_afectados_total::INTEGER,
        s.costo_estimado_total::NUMERIC(14,2)
    FROM stg_sabana s
    JOIN organizacion o
```

En resumen

El flujo del procedimiento es:

```
stg_sabana
├── organization
├── tipo_dato
├── brecha
├── usuario
└── exposicion_usuario
```

Y después de crear el procedimiento, se ejecutaría con:

```
SQL
CALL pr_poblar_modelo();
```

Importante: este procedimiento corresponde al modelo de `organization / brecha / usuario / tipo_dato` que acabas de pegar. No estoy mezclándolo con el examen anterior de Cadena de Frío.

